

tf.make_tensor_proto Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/make_tensor_proto

Create a TensorProto.

Function Signatures

```
tf.make_tensor_proto( values, dtype=None, shape=None,
verify_shape=False, allow_broadcast=False )

request =
tensorflow_serving.apis.predict_pb2.PredictRequest()
request.model_spec.name = "my_model"
request.model_spec.signature_name = "serving_default"
request.inputs["images"].CopyFrom(tf.make_tensor_proto(X_new))
```

Code Examples

```
tf.make_tensor_proto(  
    values, dtype=None, shape=None, verify_shape=False,  
    allow_broadcast=False  
)
```

```
tf.make_tensor_proto(  
    values, dtype=None, shape=None, verify_shape=False,  
    allow_broadcast=False  
)
```

```
request = tensorflow_serving.apis.predict_pb2.PredictRequest()  
request.model_spec.name = "my_model"  
request.model_spec.signature_name = "serving_default"  
request.inputs["images"].CopyFrom(tf.make_tensor_proto(X_new))
```

```
request = tensorflow_serving.apis.predict_pb2.PredictRequest()  
request.model_spec.name = "my_model"  
request.model_spec.signature_name = "serving_default"  
request.inputs["images"].CopyFrom(tf.make_tensor_proto(X_new))
```

Documentation

Create a TensorProto.

View aliases

Compat aliases for migration

See

Migration guide for

more details.

`tf.compat.v1.make_tensor_proto`

In TensorFlow 2.0, representing tensors as protos should no longer be a common workflow. That said, this utility function is still useful for generating TF Serving request protos:

`make_tensor_proto` accepts "values" of a python scalar, a python list, a numpy ndarray, or a numpy scalar.

If "values" is a python scalar or a python list, `make_tensor_proto` first convert it to numpy ndarray. If dtype is None, the conversion tries its best to infer the right numpy data type. Otherwise, the resulting numpy array has a compatible data type with the given dtype.

In either case above, the numpy ndarray (either the caller provided or the auto-converted) must have the compatible type with dtype.

`make_tensor_proto` then converts the numpy array to a tensor proto.

If "shape" is None, the resulting tensor proto represents the numpy array precisely.

Otherwise, "shape" specifies the tensor's shape and the numpy array can not have more elements than what "shape" specifies.

Returns

If values is a TensorProto, it is immediately returned; dtype and shape are ignored.

Raises